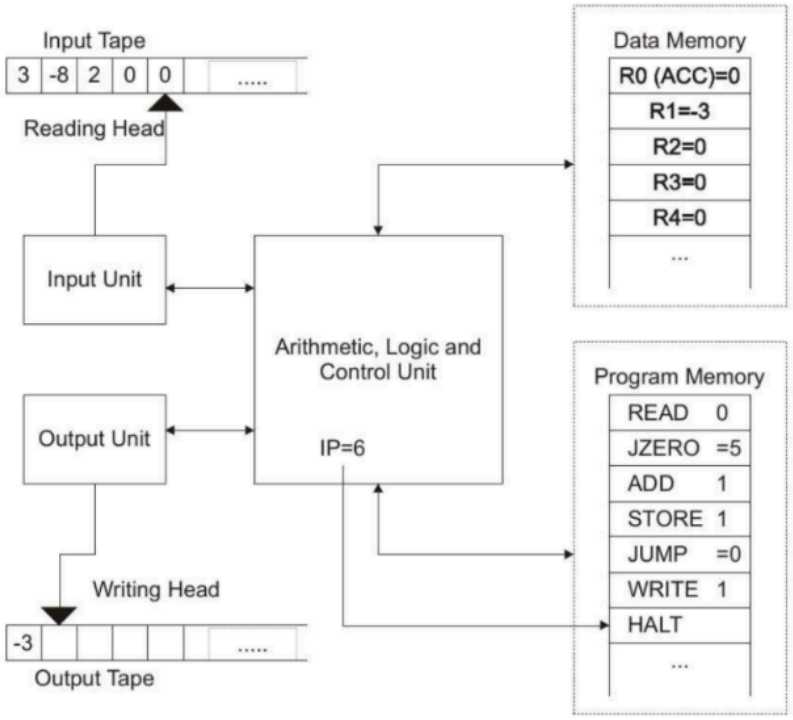
Universidad de La Laguna

Diseño y Análisis de Algoritmos

Práctica 2: Simulador de la máquina RAM



Manuel Cadenas García alu0101636849@ull.edu.es
Guillermo López Concepción alu0101620459@ull.edu.es

Fecha: 21 de febrero de 2026

C/ Padre Herrera s/n
38207 La Laguna
Santa Cruz de Tenerife. España

T: 900 43 25 26

ull.es

Implementación de la Máquina RAM

Para implementar la máquina RAM, todo se junta en la clase RAMMachine que contiene:

- Una memoria de programa: lista de punteros a instrucciones.
- Una memoria de datos: una clase DataMemory que simula registros infinitos.
- Cintas de entrada y salida: clases InputTape y OutputTape, respectivamente.
- Tabla de símbolos: guarda las distintas direcciones del programa a las que apuntan las etiquetas.
- PC: el contador del programa.

Para guardar las instrucciones, tenemos que conseguir un “ensamblador” que convierta el código a instrucciones reales que la máquina RAM pueda entender. ProgramLoader gnora comentarios y espacios en blanco, captura etiquetas, opcodes y operandos (si aparecen junto al opcode).

Con esto, la información se la pasa a InstructionCreator, que asocia cada string opcode con una instrucción real.

Al mismo tiempo, el operando es interpretado dependiendo de cómo está escrito: inmediato, directo, indirecto o vector. Esto lo realiza OperandCreator, que devuelve la estrategia de resolución correcta encapsulada en la interfaz Operand.

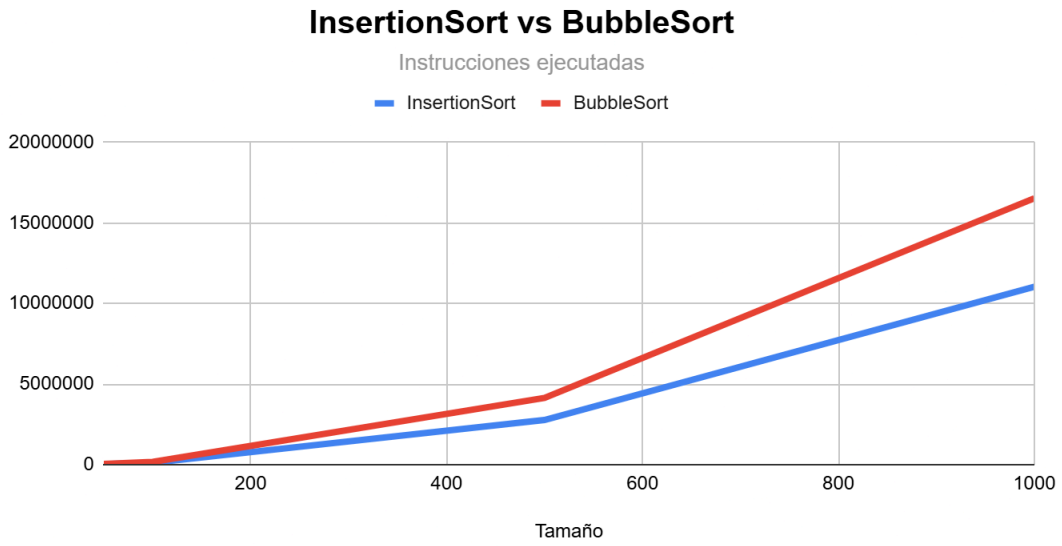
Con todo esto, la máquina se ejecuta infinitamente hasta que se le de la orden (instrucción) de parada o cuando llegue al final de la memoria del programa. Ejecuta una instrucción que no conoce realmente el tipo exacto.

Análisis de los algoritmos BubbleSort e InsertionSort con nuestra máquina RAM

Hemos creado para los algoritmos BubbleSort e InsertionSort su versión al código aceptado por la máquina RAM. Con ello, podremos medir en base al número de instrucciones ejecutadas su eficiencia respecto al tiempo.

Para ello lo mediremos con los tamaños 50, 100, 500 y hasta 1000 con los conjuntos desordenados con el peor caso: invertidos.

Tamaño	InsertionSort	BubbleSort
50	29245	42476
100	113495	167451
500	2767495	4137251
1000	11034995	16524501



Analizando los datos obtenidos y observando la gráfica, podemos afirmar que tienen un comportamiento relativamente parecido en cuanto al crecimiento, pero que InsertionSort es ligeramente mejor respecto al tiempo cuanto mayor el tamaño del vector.

Teóricamente ambos tienen un orden de complejidad de O(n^2). En la implementación práctica BubbleSort ejecuta un mayor número de instrucciones. La diferencia fundamental reside en la gestión de los intercambios.

Bubblesort realiza múltiples intercambios (swaps) en cada iteración, lo que implica operaciones constantes de carga y almacenamiento en variables temporales (DO_SWAP en nuestro código)

InsertionSort minimiza el número de escrituras, ya que solo realiza el desplazamiento final una vez encontrada la posición correcta para el elemento actual.